

Hardware Support for Operating Systems

COMS10015

Dr Tom Deakin

University of Bristol

Week 21

Operating system

- ▶ The operating system is the lowest level of software running on a processor.
- ▶ Layer of abstraction between applications we write and the hardware.
- ▶ Interface between software and Instruction Set Architecture:
 - ▶ Sets up the processor and memory.
 - ▶ Gives access to peripheral devices (Disks, GPUs, Network, ...).
 - ▶ Provides protection and security.
 - ▶ Manages memory, data and files.
- ▶ The *kernel* is the heart of the operating system providing this functionality.
 - ▶ Controls which applications run at what time: scheduling.
 - ▶ Controls Input/Output.
- ▶ Kernel instructions loaded into memory.

Kernel calls

- ▶ The kernel provides a number of routines that programs can call.
 - ▶ Usually called *system calls*.
 - ▶ Used for output, allocating memory on the heap, ...
- ▶ Processor ISA often uses *software interrupts* for this.
- ▶ Current program needs to be paused and the kernel routine executed before resuming the program.
- ▶ Software interrupts work in a manner similar to hardware interrupts, except triggered by an instruction rather than an external signal.
 - ▶ E.g. x86 INT and Arm SVC/SWI instructions.

Scheduling

Kernel Scheduler

- ▶ Kernel maintains a queue of programs.
- ▶ On interrupts (software and/or hardware), the scheduler has option to de-schedule current program and schedule another.
- ▶ Useful because if program is unable to proceed (waiting for data, input, ...), can schedule something else whilst waiting.
- ▶ Processor also shared by time, so if current program used its allotted time-slice, next program in queue is scheduled.
- ▶ When de-scheduled, the kernel:
 1. Saves the processor state (contents of registers) in memory.
 2. Takes next program from queue and restores the state (registers).
- ▶ Once waiting program ready to proceed again, it is added to back of the queue (re-schedule).

Example scheduler operation

1. **Timer interrupt:** update current time, if current program exceeded time-slice de-schedule and re-schedule it. Also re-schedule any waiting programs.
2. **Input device interrupt:** transfer data to input buffer, and if buffer is full disable interrupts.
3. **Kernel call for input:** if input buffer not empty, return the data. If input buffer empty, de-schedule the program. Enable interrupts.
4. **Output device interrupt:** transfer data to device, or if buffer is empty disable interrupts.
5. **Kernel call for output:** transfer data to output buffer, enable interrupts. If output buffer full, de-schedule.
6. **Errors:** (e.g. page fault) de-schedule program. A future interrupt will re-schedule it.
7. **Other kernel call:** e.g. file I/O, starting/ending programs, ...

At all steps, if a de-schedule program is waiting and is now ready, re-schedule it.
When a program is de-scheduled, the next one in the queue is re-instated.

Mars Pathfinder

- ▶ 1996–1998 NASA mission to land on Mars.
- ▶ Processor was an IBM Risc 6000, 128 MB RAM, 6 MB EEPROM.
- ▶ Operating Systems when scheduling have to determine whether to re-schedule the same program (non-preemptive) or another one (preemptive).
- ▶ With preemptive scheduling, can prioritise tasks.
- ▶ Challenge when tasks share data/resources; if a low priority task hold a resource and gets preempted by a medium priority tasks, a high priority tasks needing that resource will have to wait: causes **priority inversion**.
- ▶ On Pathfinder, the lander collected weather data (a low priority task), causing a hold on the bus.
- ▶ Communications (higher priority) would be scheduled, and the OS would detect the bus wasn't being used and reset the OS.
- ▶ Remotely fixed by uploading a C program from Earth and ran using the onboard C interpreter to enable priority inheritance.

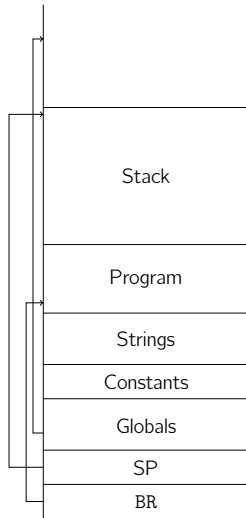
Loaders

Loading executables

- ▶ Executable stored on disk.
- ▶ Needs to be brought into memory for execution.
- ▶ A *loader* does the job of reading the executable and placing it in memory.
- ▶ Loader typically provided by the operating system.
- ▶ Place any program arguments on the stack.
- ▶ Initialise registers such as the stack pointer.
- ▶ Provide kernel with memory address of program start point.

Executable formats

- ▶ Recall: lecture on stacks.
- ▶ When program placed in memory, has a specific structure.
- ▶ Compiler/Assembler and linker and loader need to all cooperate.
- ▶ E.g. Executable and Linkable Format (ELF).
 - ▶ ELF is a standard binary format which explains structure of the binary.
 - ▶ Along with metadata that describes the location/size of those sections.
 - ▶ Contains the machine code, data, external symbols, ...



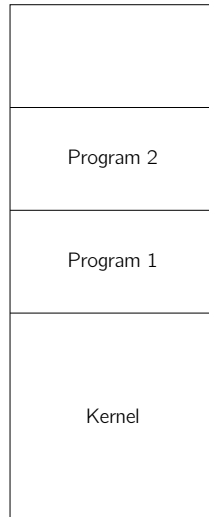
Dynamic linking loader

- ▶ Modern operating systems allow *dynamic linking*.
- ▶ Allows for some references to be resolved at **runtime**.
- ▶ Executable still has some unresolved references, which are resolved when the program is *loaded*.
- ▶ E.g. `.dll` on Windows and `.so` on Linux.
- ▶ Can be useful to try out different libraries which provide the same references.
- ▶ In Linux, `LD_PRELOAD` and `LD_LIBRARY_PATH` environment variables.

Protection

Memory protection

- ▶ Processor should ensure programs do not interfere with each other.
- ▶ Kernels and programs reside in different parts of memory at the same time.
- ▶ One program should not be able to access the data/instructions of another program, or the kernel itself.
- ▶ Kernel is allowed to access full memory space.
- ▶ But programs should only be able to access own part of memory.

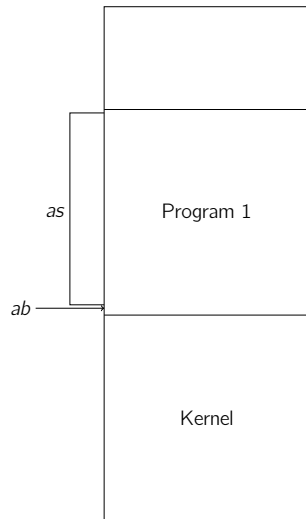


Supporting privilege modes

- ▶ Need a way for processor to operate in different modes:
 - ▶ User mode vs supervisor mode.
 - ▶ And a means to track this, e.g., Armv7-A Mode bits in CPSR.
 - ▶ Sometimes called Protection Rings.
- ▶ Designate a portion of memory as protected.
 - ▶ Often readable by all processes.
 - ▶ Writable only when in supervisor mode, using special instructions.
- ▶ Mechanism to switch between user and supervisor mode, and vice versa.
 - User-to-supervisor System calls through software interrupts.
 - Supervisor-to-user Returning from interrupts updates the mode.

Memory region protection

- ▶ Add two special-purpose registers to the ISA.
 - ab application base
 - as application size
- ▶ These are set by the operating system when a program starts (on Load).
- ▶ Flag bit *ink* to record if executing kernel (Privileged execution).
- ▶ If executing kernel, $ink = 1$ and all memory is available.
- ▶ Otherwise, only allowed to access memory in range $[ab, ab + as]$.



Memory region protection: new instructions

Two new instructions are added to set those registers:

SETAB If *ink* is 1 then $ab \leftarrow oreg$ else error.

SETAS If *ink* is 1 then $as \leftarrow oreg$ else error.

- ▶ Instructions only available to be executed by the kernel.
- ▶ Applications not allowed to choose position in memory — the Operating System sets these.
- ▶ Register values saved as part of program state when kernel de-schedules.

Memory region protection: data access instructions

- ▶ In Hex 8 ISA, we have data access instructions **LDAM**, **STAM**, **LDAI**, **STAI**.
- ▶ E.g. **STAM** $mem[oreg] \leftarrow areg$
- ▶ If instruction part of a program, need to ensure *oreg* is within the valid application memory range.
- ▶ Update the implementation:

STAM if *ink* is 1 then $mem[oreg] \leftarrow areg$
 else if $oreg < as$ then $mem[ab + oreg] \leftarrow areg$
 else error
- ▶ Offsets (in operands) are relative to the application base *ab*.
- ▶ Each application written assuming memory addresses start at zero — recall Position Independent Code.

Memory region protection: branch instructions

- ▶ In Hex 8 ISA, we have branch instructions **BR**, **BRZ**, **BRN**, **BRB**.
- ▶ E.g. **BR** $pc \leftarrow pc + oreg$
- ▶ As with data access, need to ensure *oreg* is within application memory range.
- ▶ Update the implementation:
 BR if *ink* is 1 then $pc \leftarrow pc + oreg$
 else if $(pc + oreg) < as$ then $pc \leftarrow pc + oreg$
 else error

Next time: Virtual memory

- ▶ Can also implement memory protection using virtual memory.
- ▶ That's the topic of the next lecture.
- ▶ Still requires a portion of memory that is only accessible when in supervisor mode.

Further Reading

- ▶ https://en.wikipedia.org/wiki/System_call
- ▶ The First Bug on Mars: OS Scheduling, Priority Inversion, and the Mars Pathfinder <https://kwahome.medium.com/the-first-bug-on-mars-os-scheduling-priority-inversion-and-the-mars-pathfinder-53586a631525>
- ▶ What really happened on Mars Rover Pathfinder <https://www.cs.cornell.edu/courses/cs614/1999sp/papers/pathfinder.html>
- ▶ John R. Levine, Linkers & Loaders.
- ▶ [https://en.wikipedia.org/wiki/Loader_\(computing\)](https://en.wikipedia.org/wiki/Loader_(computing))

Further Reading

- ▶ https://en.wikipedia.org/wiki/Executable_and_Linkable_Format
- ▶ https://linuxhint.com/understanding_elf_file_format/
- ▶ Linkers and Loaders, <https://www.linuxjournal.com/article/6463>
- ▶ Chapter 7, Object Codes, Loaders and Linkers
<http://homepage.divms.uiowa.edu/~jones/syssoft/notes/07link.html>
- ▶ Linux x86 Program Start Up—or—How the heck do we get to main()? <http://dbp-consulting.com/tutorials/debugging/linuxProgramStartup.html>